# Software Requirements Specification

for

## Credit Card Fraud Detection and Model Performance Evaluation System

Version 1.2 (SRS-CCFD-1.2)

Prepared by: Team Error 404

Date: 12-Oct-2026

Technology: Python Flask, REST APIs, HTML, CSS, JavaScript

Operating Environment: Windows or Linux

Architecture: Web Client → Web Server → ML / DL / QML Microservices

# Table of Contents


- Revision History
- 1. Introduction
  - 1.1 Purpose
  - 1.2 Document Conventions
  - 1.3 Intended Audience and Reading Suggestions
  - 1.4 Product Scope
  - 1.5 References
- 2. Overall Description
  - 2.1 Product Perspective
  - 2.2 Product Functions
  - 2.3 User Classes and Characteristics
  - 2.4 Operating Environment
  - 2.5 Design and Implementation Constraints
  - 2.6 User Documentation
  - 2.7 Assumptions and Dependencies
- 3. External Interface Requirements
  - 3.1 User Interfaces
  - 3.2 Hardware Interfaces
  - 3.3 Software Interfaces
  - 3.4 Communications Interfaces
- 4. System Features
  - 4.1 Dataset Upload and Validation
  - 4.2 Model Selection
  - 4.3 Fraud Detection Execution
  - 4.4 Model Performance Evaluation
  - 4.5 Results Presentation
  - 4.6 REST API Operations
- 5. Other Nonfunctional Requirements
  - 5.1 Performance Requirements
  - 5.2 Safety Requirements
  - 5.3 Security Requirements
  - 5.4 Software Quality Attributes
  - 5.5 Business Rules
- 6. Other Requirements
- Appendix A: Glossary
- Appendix B: Analysis Models
- Appendix C: To Be Determined List
- Project Team

# Revision History

| Name | Date | Reason for Changes | Version |
|---|---|---|---|
| Team Error 404 | 02-Sep-2026 | Initial SRS for the credit card fraud detection product. | 1.0 |
| Team Error 404 | 11-Oct-2026 | Refined per the SE mini-project guidelines: rewrote all requirements to be clear, unambiguous, concise and measurable; completed the security section with security objectives and security requirements; added the UML use case diagram; resolved TBD items for dataset schema, baseline models, performance metrics and API authentication. | 1.1 |
| Team Error 404 | 12-Oct-2026 | Consistency review with the Test Plan and the Architecture & Design Specification: separated the upload timing targets (NFR-01, NFR-03, new NFR-07) and defined the point from which they are measured; stated that microservices read the uploaded file from the dataset store by dataset_id; defined the API-key, results-retrieval and data-display (masking) policies; defined the 24-hour retention period; gave quality attributes IDs and priorities; clarified REQ-FD-03, REQ-FD-12 and REQ-FD-20; corrected stimulus/response and figure numbering; added Section 1.5. | 1.2 |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for Release 1.0 of the Credit Card Fraud Detection and Model Performance Evaluation System, a web-based product that lets a user upload a labeled credit-card transaction dataset, run fraud detection with a selected Machine Learning (ML), Deep Learning (DL) or Quantum Machine Learning (QML) model served through microservices, and view per-transaction predictions together with standard model-performance metrics.

This SRS covers the complete Release 1.0 scope: dataset upload and validation, model selection, asynchronous fraud-detection execution, performance evaluation, results presentation, and the REST API between the Web Client, the Flask Web Server and the model microservices.

Out of scope for this release: mobile applications, real-time payment authorization or transaction blocking, model retraining pipelines, multi-host deployment (TBD-01) and production hardening for public-internet deployment (TBD-02).

## 1.2 Document Conventions

- The keywords *shall* and *must* indicate mandatory requirements; *should* indicates recommended behavior.
- Each requirement carries a unique identifier:
  - REQ-FD-nn — functional requirements (fraud-detection features)
  - REQ-UI-nn — user interface requirements
  - REQ-API-nn — REST API requirements
  - NFR-nn — performance and capacity requirements
  - NFR-Snn — safety requirements
  - NFR-Qnn — software quality attributes

    - SO-nn — security objectives
    - SR-nn — security requirements
    - BR-nn — business rules
- Every REQ, NFR and SR requirement carries a priority: High, Medium or Low. Security objectives (SO) state goals that the SR requirements realize, and business rules (BR) are mandatory constraints; neither carries a separate priority.
- Every quantitative target states the condition under which it is measured (file size or row count, concurrency level, reference machine). Upload timing targets are measured from *upload completion*, defined in Section 5.1.

## 1.3 Intended Audience and Reading Suggestions

- **Developers:** Sections 2, 3 and 4 for architecture, interfaces, constraints and the functional requirements to implement.
- **Testers:** Sections 4 and 5 to derive functional and nonfunctional test cases; requirement identifiers support traceability to the Software Test Plan (STP-CCFD-1.1).
- **Faculty evaluators / project managers:** Sections 1, 2 and 5 for scope, objectives, dependencies and quality/security expectations.
- **End users:** Sections 3.1 and 4.5, together with the user manual (Section 2.6), to operate the system.

Readers should review Sections 1–2 first for context, then Section 3 for interfaces, Section 4 for the functional features in execution order, and Section 5 for the nonfunctional requirements.

## 1.4 Product Scope

The system provides a single web interface for running and comparing credit-card fraud-detection models. A user uploads a labeled transaction dataset through the Web Client, selects one model-processing option (ML, DL or QML) and starts an analysis run. The Flask Web Server stores the dataset and sends the dataset identifier to the corresponding model microservice, which reads the dataset from the shared dataset store, classifies each transaction and returns predictions with fraud probabilities. The Web Client then presents the per-transaction predictions and the model's performance metrics (accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix).

Objectives: (1) accept labeled transaction datasets in a documented, validated format; (2) execute the user-selected model through an isolated microservice; (3) report predictions and comparable performance metrics for every run. The modular microservice design keeps each model independently deployable, replaceable and testable, and gives the project a reproducible basis for comparing ML, DL and QML approaches to fraud detection.

## 1.5 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
2. SE Mini-Project Deliverables Part-1 — course submission guidelines.
3. Software Test Plan, Credit Card Fraud Detection and Model Performance Evaluation System, v1.1 (STP-CCFD-1.1).
4. Software Architecture & Design Specification, same system, v1.1 (SADS-CCFD-1.1).
5. RFC 9110, *HTTP Semantics* (status-code meanings used in REQ-API-04).

# 2. Overall Description

## 2.1 Product Perspective

This product is a new, self-contained system; it does not replace an existing fraud-detection product. It follows a three-tier architecture:

Web Client (browser) → Flask Web Server (REST API) → ML / DL / QML model microservices

The Web Client communicates with the Python Flask Web Server over HTTP using REST APIs and JSON payloads. The Web Server validates input, enforces API-key authentication on all REST API endpoints, stores accepted datasets in a server-local dataset store, and routes analysis jobs to the selected model microservice. Each microservice exposes a uniform HTTP contract (POST /predict, GET /health): it receives only a dataset identifier, reads the corresponding CSV file from the dataset store (read-only) and returns predictions with per-transaction fraud probabilities. Dataset contents are never sent inside API or microservice request bodies. No external database or payment system is involved. Figure 2 in Appendix B.2 shows the major components and their interconnections.

## 2.2 Product Functions

- Provide a browser-based Web Client for dataset upload, model selection, run control and results viewing.
- Accept and validate a labeled credit-card transaction dataset (CSV, documented schema).
- Expose REST API endpoints for dataset submission, analysis execution and status/result retrieval.
- Route each analysis request to the user-selected ML, DL or QML model microservice.
- Execute fraud detection asynchronously and report job status (queued, running, completed, failed).
- Compute standard performance metrics (accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix) for the executed model.
- Display per-transaction predictions and model-performance results; support CSV download of predictions.
- Provide user documentation and in-app guidance for operating the product.

## 2.3 User Classes and Characteristics

- **General User (primary):** uploads a dataset, selects a model type, runs fraud detection, and interprets predictions and metrics. Typical profile: undergraduate student or analyst; uses the system per evaluation session through the Web Client only. All primary functional requirements serve this class.
- **Developer / Tester (secondary):** exercises the REST APIs and model microservices directly (for example with curl or an API client) to verify contracts, run integration tests and diagnose failures. Higher technical expertise; infrequent but critical use. API and testability requirements serve this class.

There are no tiered privilege levels in Release 1.0: every caller of the REST API, including the Web Client, presents the same kind of API key (SR-03).

## 2.4 Operating Environment

- **Server:** Windows 10/11 or Ubuntu 20.04+ on x86-64; Python 3.10+; Flask 3.x; minimum 8 GB RAM and 2 GB free disk space.
- **Client:** a current version of Google Chrome, Mozilla Firefox or Microsoft Edge with JavaScript enabled.

- **Model services:** each microservice runs as an independent HTTP process on the **same host** as the Web Server, under the same operating-system service account, so that it can read the dataset store (SR-05). Multi-host deployment with a shared dataset store is TBD-01.
- **Network:** TCP/IP; HTTP between Web Client, Web Server and microservices. Release 1.0 targets a single host; the Web Client may be opened from that host or from a trusted LAN (communication security beyond the host is TBD-02).

## 2.5 Design and Implementation Constraints

- The Web Server shall be implemented in Python using the Flask framework.
- All client–server and server–microservice communication shall use HTTP with JSON payloads.
- ML, DL and QML functionality shall be exposed as separate HTTP microservices behind the uniform contract defined in Section 3.3.
- The Web Client shall be implemented with HTML, CSS and JavaScript only (no front-end build toolchain).
- QML models shall execute on a classical quantum-circuit simulator; no physical quantum hardware is required or assumed.
- The system shall use file-system storage for datasets, job records and results; no database management system shall be required.

## 2.6 User Documentation

The following documentation shall be delivered with the software:

1. User manual (PDF and/or HTML): how to open the Web Client and enter the API key, the dataset schema (required Class column and the feature columns expected by the trained models), how to upload a dataset, select a model, start and monitor a run, interpret predictions and metrics, download results, and troubleshoot common errors.
2. In-application guidance: labeled controls, tooltips, inline validation messages and a help link visible on every screen.
3. Developer README: microservice contracts, API-key configuration, the configured classifier of each microservice, and instructions for running the Web Server and the three model services.

## 2.7 Assumptions and Dependencies

- **A1.** The user supplies a labeled dataset conforming to the CSV schema defined in REQ-FD-02 and REQ-FD-03.
- **A2.** The Flask Web Server and all three model microservices are running on the same host when an analysis is requested.
- **A3.** The host running the QML microservice has the required quantum-circuit simulator libraries installed.
- **A4.** The baseline models are trained before deployment (model training is outside the product); the feature columns they expect are listed in the user manual.
- **D1.** The project depends on the Python/Flask ecosystem and numerical/ML libraries available in the deployment environment; exact library versions are pinned in the developer README.
- **D2.** Quantitative performance targets in Section 5.1 are measured on the reference machine defined in Section 2.4 (x86-64, 8 GB RAM) unless stated otherwise.

# 3. External Interface Requirements

## 3.1 User Interfaces

The Web Client is a single-page browser interface organized into four regions: dataset upload (with the API-key field), model selection, run control with live status, and results (predictions table plus performance metrics). Detailed screen design is documented in the SADS (Section 3.5); the following requirements govern behavior:

- **REQ-UI-01 (High):** The GUI shall be implemented using HTML, CSS and JavaScript, and shall provide, on one page, an API-key field and controls for dataset upload, model selection, analysis execution and results viewing.
- **REQ-UI-02 (High):** The system shall display an error message adjacent to the control that caused it within 1 second of receiving the server's error response.
- **REQ-UI-03 (High):** The run-control region shall show the current job status — queued, running, completed or failed — and shall update it without requiring a page reload.
- **REQ-UI-04 (Medium):** A help link opening the user manual shall be visible on every screen.
- **REQ-UI-05 (Medium):** Prediction tables shall be paginated at 50 rows per page.
- **REQ-UI-06 (Low):** The interface shall use a single stylesheet so that controls, messages and tables render consistently across screens, and shall remain usable at a viewport of 1280×720 and above.

## 3.2 Hardware Interfaces

No specialized hardware interface is required. The product runs on a general-purpose Windows or Linux computer meeting the minimum configuration in Section 2.4 (x86-64 CPU, 8 GB RAM, 2 GB free disk), using the standard keyboard, mouse and display interfaces provided by the client operating system.

## 3.3 Software Interfaces

- **Web Client ↔ Web Server:** REST over HTTP; request and response bodies use application/json with UTF-8 encoding, except the dataset upload, which uses multipart/form-data. The endpoint contract is defined in Section 4.6.
- **Web Server ↔ model microservices:** each microservice (ML, DL, QML) shall expose:
  - `POST /predict` — accepts `{ "dataset_id": "<id>" }`; reads the file for that identifier from the dataset store; returns HTTP 200 with `{ "model_name": "<name>", "predictions": [ { "row": <int>, "class": 0|1, "probability": <0..1> } ], "metrics": { "accuracy", "precision", "recall", "f1", "roc_auc", "confusion_matrix": { "tp", "tn", "fp", "fn" } } }`; returns HTTP 400 if the request body is malformed, HTTP 404 if the dataset identifier is unknown and HTTP 500 if the model fails, each with the error body defined in REQ-API-04.
  - `GET /health` — returns HTTP 200 with `{ "status": "ok" }` when the service is ready to accept work, otherwise HTTP 503.
- **Dataset store:** a server-local directory holding uploaded CSV files, job records and result files. The Web Server reads and writes it; the microservices only read dataset files. Access control and retention are defined in SR-05.
- **External systems:** none. The product does not interface with payment gateways, banking systems or identity providers in Release 1.0.

## 3.4 Communications Interfaces

- All communication between the Web Client, Web Server and microservices shall use HTTP/1.1 over TCP/IP with JSON message bodies (multipart/form-data for the dataset upload).

- The Web Client shall poll `GET /api/analyses/{job_id}` at 2-second intervals while a job is in the queued or running state, and shall stop polling once the job reaches a terminal state (completed or failed).
- The model microservices shall accept connections only from the local host (loopback interface); they are not reachable from the Web Client.
- HTTP status codes shall follow RFC 9110 semantics as specified in REQ-API-04.
- Communication security for access from other machines (TLS termination) is tracked as TBD-02 in Appendix C.

# 4. System Features

The functional requirements are organized by system feature, following the execution order of the primary use case (see the UML use case diagram, Figure 1, Appendix B.1).

## 4.1 Dataset Upload and Validation

**Description and Priority:** The system accepts a labeled credit-card transaction dataset through the Web Client, validates it against the supported schema, and either registers it for analysis or rejects it with a specific, understandable error. Priority: High.

**Stimulus/Response Sequences:**

1. The user enters the API key, opens the upload control and selects a dataset file.
2. The Web Client submits the file to the Flask Web Server via `POST /api/datasets`.
3. The Web Server checks the API key, validates file type, size and schema, then either stores the file and returns HTTP 201 with a dataset identifier, or returns HTTP 400 naming the reason for rejection.
4. The Web Client shows a success confirmation or the returned error message next to the upload control.

**Functional Requirements:**

- **REQ-FD-01 (High):** The system shall provide a file-upload control in the Web Client that accepts a single dataset file per submission.
- **REQ-FD-02 (High):** The system shall accept files in CSV format — UTF-8 encoded, comma-delimited, with a header row — with a size of at most 500 MB and at most 1,000,000 data rows.
- **REQ-FD-03 (High):** The system shall validate that the uploaded file contains at least one data row, a fraud-label column named `Class` whose values are restricted to 0 (legitimate) and 1 (fraud), and every feature column expected by the trained models as listed in the documented dataset schema (user manual); all values in those feature columns shall be numeric. Additional columns are permitted and are ignored by the models. (Validation time limits are given in NFR-01, NFR-03 and NFR-07.)
- **REQ-FD-04 (High):** The system shall reject an invalid, missing or unsupported upload and display an error message that names the reason (for example: "unsupported file type", "missing required column: Class", "file exceeds the 500 MB limit").
- **REQ-FD-05 (Medium):** The system shall assign a unique dataset identifier to each accepted upload and use it to associate all subsequent analysis results for the duration of the dataset's retention period (SR-05).

## 4.2 Model Selection

**Description and Priority:** The system offers the three model-processing options (ML, DL, QML), each backed by an independent microservice, and lets the user choose exactly one per analysis run. Priority: High.

**Stimulus/Response Sequences:**

1. The user views the available model options in the Web Client.
2. The user selects one of ML, DL or QML.
3. The Web Client records the selection and enables the run control.

**Functional Requirements:**

- **REQ-FD-06 (High):** The system shall offer exactly the three model-processing options ML, DL and QML, and shall require the user to select exactly one option before an analysis run can start.
- **REQ-FD-07 (High):** The ML microservice shall provide trained machine-learning classifiers (baseline: Logistic Regression and Random Forest) and shall run the one classifier named in its configuration for each analysis.
- **REQ-FD-08 (High):** The DL microservice shall expose at least one trained deep-learning classifier (baseline: multilayer perceptron).
- **REQ-FD-09 (High):** The QML microservice shall expose at least one quantum machine-learning classifier executed on a classical quantum-circuit simulator.
- **REQ-FD-10 (Medium):** Each model microservice shall accept a dataset identifier, read the corresponding dataset from the dataset store, and return predictions with per-transaction fraud probabilities through the uniform HTTP contract defined in Section 3.3.

## 4.3 Fraud Detection Execution

**Description and Priority:** The Web Server routes each validated analysis request to the selected model microservice, executes it as an asynchronous job, and reports completion or failure. Priority: High.

**Stimulus/Response Sequences:**

1. The user, having an accepted dataset and a selected model option, activates the run control.
2. The Web Client sends `POST /api/analyses` with the dataset identifier and model type.
3. The Web Server checks the API key and the request, creates a job in the queued state, and returns HTTP 202 with a job identifier.
4. A background worker of the Web Server marks the job running and calls `POST /predict` on the selected microservice with the dataset identifier; the microservice reads the dataset from the dataset store.
5. On a valid response, the job is marked completed and its predictions and metrics are stored; on microservice error, invalid response or timeout, the job is marked failed with a recorded reason.
6. The Web Client polls the job status and displays the final outcome.

**Functional Requirements:**

- **REQ-FD-11 (High):** The Web Server shall route each analysis request to the microservice corresponding to the model option selected by the user.
- **REQ-FD-12 (High):** The system shall execute fraud detection as an asynchronous job: the server shall return HTTP 202 with a job identifier within 2 seconds of receiving a valid request, and the job shall transition through the states queued → running → completed or failed.
- **REQ-FD-13 (High):** The system shall classify every transaction row of the accepted dataset and return, for each row, the predicted class (0 = legitimate, 1 = fraud) and the fraud probability in the range [0, 1].
- **REQ-FD-14 (High):** If the selected microservice returns an error, returns an invalid response, cannot be reached, or produces no result within 600 seconds of the job entering the running state, the system shall mark the job as failed with a recorded reason and report the failure to the user; a failed job shall never be presented as a successful result.

## 4.4 Model Performance Evaluation

**Description and Priority:** After a successful run, the system computes standard classification metrics for the executed model so that ML, DL and QML approaches can be compared on the same dataset. Priority: High.

**Stimulus/Response Sequences:**

1. The model microservice completes classification of the dataset.
2. The microservice computes performance metrics against the Class label column and returns them with the predictions and the model name.
3. The Web Server stores the metrics with the job record.
4. The Web Client displays the metrics labeled with the executed model's name.

**Functional Requirements:**

- **REQ-FD-15 (High):** After a successful run, the system shall compute and return accuracy, precision, recall, F1-score and ROC-AUC for the executed model, each reported to four decimal places.
- **REQ-FD-16 (High):** The system shall return the confusion-matrix counts — true positives, true negatives, false positives and false negatives — for the executed model.
- **REQ-FD-17 (Medium):** The system shall label every displayed metric with the name of the model that produced it.

## 4.5 Results Presentation

**Description and Priority:** The Web Client presents the fraud-detection output and model-performance results of the completed job and lets the user download the predictions. Priority: High.

**Stimulus/Response Sequences:**

1. The polled job reaches a terminal state.
2. For a completed job, the Web Client displays the predictions table, the metrics and the confusion matrix; for a failed job, it displays the recorded reason.
3. The user pages through the predictions and, optionally, downloads them as a CSV file.

**Functional Requirements:**

- **REQ-FD-18 (High):** The Web Client shall display the per-transaction predictions in a paginated table showing the row index, predicted class and fraud probability.
- **REQ-FD-19 (High):** The Web Client shall display the performance metrics and confusion matrix returned for the executed model.
- **REQ-FD-20 (High):** When a job reaches a terminal state, the Web Client shall, without a page reload or further user action, display the results (completed) or the recorded failure reason (failed).
- **REQ-FD-21 (Medium):** The system shall allow the user to download the prediction results as a CSV file containing the row index, predicted class and fraud probability.
- **REQ-FD-22 (Medium):** The system shall keep each result set associated with the dataset identifier and model that generated it for the duration of the dataset's retention period (SR-05).

## 4.6 REST API Operations

**Description and Priority:** The REST API is the service boundary between the Web Client and the Web Server and supports direct invocation by developers and testers. Priority: High.

**Stimulus/Response Sequences:**

1. A client (the Web Client or a developer's API client) sends an HTTP request with the API key.

2. The Web Server returns a JSON response with the status code defined in REQ-API-04.

**Functional Requirements:**

- **REQ-API-01 (High):** The Web Server shall expose `POST /api/datasets` for dataset upload; on success it shall return HTTP 201 with a JSON body containing the dataset identifier.
- **REQ-API-02 (High):** The Web Server shall expose `POST /api/analyses`, accepting `{ "dataset_id": "<id>", "model_type": "ml" | "dl" | "qml" }`; on acceptance it shall return HTTP 202 with a JSON body containing the job identifier and the status "queued".
- **REQ-API-03 (High):** The Web Server shall expose `GET /api/analyses/{job_id}`, returning the job status and, when the job is completed, the predictions, performance metrics and model name, or, when the job has failed, the recorded reason.
- **REQ-API-04 (High):** The Web Server shall use standard HTTP status codes — 200 (OK), 201 (created), 202 (accepted), 400 (invalid request), 401 (missing or invalid API key), 404 (unknown dataset or job), 405 (method not allowed), 500 (server error) — and every error response shall carry a JSON body containing a machine-readable error code and a human-readable message.
- **REQ-API-05 (High):** All API routes shall be implemented as Python Flask routes returning JSON response bodies.

# 5. Other Nonfunctional Requirements

## 5.1 Performance Requirements

*Upload completion* is the moment the Web Server has received the complete upload request body. Network transfer time is excluded from every upload target below, because it depends on the user's connection rather than on the system.

- **NFR-01 (High):** For a dataset of up to 10,000 data rows, the Web Server shall return the upload result (HTTP 201 or HTTP 400) within 2 seconds of upload completion, on a single-host deployment on the reference machine.
- **NFR-02 (High):** The job-status endpoint `GET /api/analyses/{job_id}` shall respond within 500 ms at the 95th percentile with 10 concurrent polling clients.
- **NFR-03 (Medium):** For a valid 100,000-row dataset, full validation and storage shall complete, and HTTP 201 shall be returned, within 60 seconds of upload completion on the reference machine (x86-64, 8 GB RAM).
- **NFR-04 (Medium):** The results page shall render within 3 seconds of receiving a completed job response for a dataset of up to 10,000 rows.
- **NFR-05 (Medium):** The system shall support at least 10 concurrent analysis sessions without violating NFR-01, NFR-02 or NFR-04.
- **NFR-06 (Low):** Model microservices shall be independently deployable: stopping any one service shall not prevent the other two from serving requests.
- **NFR-07 (Medium):** For any file up to the 500 MB limit that fails a file-level check — unsupported file type, size above the limit, wrong delimiter, or a missing required column — the Web Server shall return HTTP 400 within 5 seconds of upload completion, independent of the number of rows.

Rationale: NFR-01 covers the typical upload, NFR-07 guarantees fast rejection of structurally wrong files of any size (these checks need only the file size and header), and NFR-03 bounds full row-level validation of a large file. Row-level validation of files larger than 100,000 rows is not time-bounded in Release 1.0. Model training/inference time is data- and hardware-dependent and is therefore not bounded either; the asynchronous job model (REQ-FD-12) with live status reporting keeps the system responsive regardless of run duration, and REQ-FD-14 caps a single run at 600 seconds.

## 5.2 Safety Requirements

- **NFR-S01 (High):** The system shall not initiate, authorize or block any real financial transaction; all outputs are analytical.
- **NFR-S02 (High):** Every results screen shall display the notice: "Analytical result only — not a financial authorization decision."
- **NFR-S03 (Medium):** Any sample or demonstration dataset shipped with the system shall contain no real cardholder data.

## 5.3 Security Requirements

### 5.3.1 Security Objectives

- **SO-01 — Confidentiality of transaction data:** uploaded datasets and the results derived from them shall be protected from unauthorized access or exposure throughout upload, processing, storage and display.
- **SO-02 — Authorized service invocation:** only clients presenting valid credentials shall be able to upload datasets, trigger model execution or retrieve results.
- **SO-03 — Input trustworthiness:** untrusted input (uploaded files, JSON payloads) shall not be able to compromise the Web Server or the model microservices.

### 5.3.2 Security Requirements

- **SR-01 (High):** The system shall validate file type, size and schema before any model processing begins; non-conforming uploads shall be rejected with HTTP 400, shall not be stored, and shall never reach a model microservice. (Supports SO-01, SO-03.)
- **SR-02 (High):** All JSON request bodies shall be parsed with strict schema validation (required fields present, no unknown fields, model_type restricted to "ml", "dl" or "qml"); malformed payloads shall be rejected with HTTP 400, and the payload contents shall not be written to logs. (Supports SO-03.)
- **SR-03 (High):** Every REST API endpoint — `POST /api/datasets`, `POST /api/analyses` and `GET /api/analyses/{job_id}` — shall require a valid API key presented in the HTTP Authorization header; requests with a missing or invalid key shall receive HTTP 401 and no processing shall occur. The Web Client shall obtain the key from the user through its API-key field and keep it only in browser memory for the open page (not in cookies or browser storage). Job identifiers shall be randomly generated so that they cannot be guessed. (Supports SO-01, SO-02.)
- **SR-04 (High):** The system shall not write transaction field values to application logs. The Web Client and the CSV download shall show, per transaction, only the row index, predicted class and fraud probability, plus aggregate metrics; no other column of the uploaded dataset shall be displayed or exported, and error messages may name a column or row number but shall not echo cell values. (Supports SO-01.)
- **SR-05 (Medium):** Uploaded datasets, job records and result files shall be stored in a dedicated server-side directory whose permissions allow access only by the operating-system account that runs the Web Server and the microservices. Each dataset, together with its job records and results, shall be deleted when its retention period ends, 24 hours after upload. (Supports SO-01.)
- **SR-06 (Medium):** Error responses sent to clients shall not disclose stack traces, file-system paths or internal configuration values. (Supports SO-01, SO-02.)

## 5.4 Software Quality Attributes

- **NFR-Q01 Usability (Medium):** A first-time user shall be able to complete the full workflow — enter the API key, upload a valid dataset, select a model, run analysis and view results — within 5 minutes without assistance, as measured in a usability walkthrough.

- **NFR-Q02 Maintainability (Medium):** Each model microservice shall be replaceable independently through the uniform HTTP contract defined in Section 3.3, without changes to the Web Server or Web Client.
- **NFR-Q03 Reliability (Medium):** Under the load stated in NFR-05, at least 99% of valid API requests shall receive a response with the correct status code.
- **NFR-Q04 Portability (Low):** The product shall operate on Windows 10/11 and Ubuntu 20.04+ as specified in Section 2.4, with all platform differences isolated in deployment configuration.
- **NFR-Q05 Testability (Medium):** Each model microservice shall be testable in isolation through its HTTP contract (POST /predict, GET /health).

### 5.5 Business Rules

- **BR-01:** Only datasets intended for credit-card fraud analysis, conforming to the schema in REQ-FD-02 and REQ-FD-03, shall be submitted for analysis.
- **BR-02:** A model option (ML, DL or QML) must be selected before an analysis request is executed.
- **BR-03:** The system shall display performance metrics only for the model that was actually executed in the corresponding job.
- **BR-04:** A failed model-service request shall never be presented as a successful fraud-detection result.
- **BR-05:** All predictions and metrics shall be presented as analytical results, never as authorization decisions (see NFR-S02).

## 6. Other Requirements

- **Data storage:** No database management system is required; datasets, job records and results are held as files in the server-local dataset store (see SR-05 for access control and retention).
- **Internationalization:** The user interface and documentation shall be in English; no other locale support is required in Release 1.0.
- **Legal and privacy:** The system shall not be used with real cardholder data in Release 1.0; demonstration datasets must be synthetic or anonymized (see NFR-S03).
- **Reuse objectives:** The uniform microservice contract (Section 3.3) is designed so that additional models can be added as new microservices without modifying the Web Client; the Web Server needs only a configuration entry mapping the new model type to the service address.

## Appendix A: Glossary

| Term | Meaning |
|---|---|
| Web Client | Browser-based GUI (HTML, CSS, JavaScript) through which the user supplies datasets, selects models and views results. |
| Web Server | Python Flask server that exposes the REST API, validates input, enforces authentication, stores datasets and routes jobs to model microservices. |
| REST API | HTTP-based interface between the Web Client and the Web Server, using JSON payloads and standard status codes. |
| ML | Machine Learning — classical statistical learning models (baseline: Logistic Regression, Random Forest). |
| DL | Deep Learning — neural-network models (baseline: multilayer perceptron). |

| Term | Meaning |
|---|---|
| QML | Quantum Machine Learning — quantum classifiers executed on a classical quantum-circuit simulator. |
| Microservice | Independently deployable HTTP service responsible for one model family, exposing POST /predict and GET /health. |
| Dataset | A labeled CSV file of credit-card transactions with a binary Class column (0 = legitimate, 1 = fraud) and the documented feature columns. |
| Dataset store | Server-local directory holding uploaded datasets, job records and result files. |
| Dataset identifier (dataset_id) | Unique identifier returned for an accepted upload; microservices use it to locate the dataset file. |
| Job | One asynchronous analysis run: a dataset identifier plus a selected model type, tracked through queued → running → completed/failed. |
| Retention period | The 24 hours after upload during which a dataset, its jobs and its results are kept; they are deleted when it ends (SR-05). |
| Upload completion | The moment the Web Server has received the complete upload request body; the reference point for upload timing targets. |
| Fraud Detection | Classification of credit-card transactions as legitimate or potentially fraudulent by the selected model. |
| Accuracy / Precision / Recall / F1-score | Standard classification metrics computed against the Class label column. |
| ROC-AUC | Area under the receiver operating characteristic curve; measures ranking quality of fraud probabilities. |
| Confusion Matrix | Counts of true positives, true negatives, false positives and false negatives for a run. |
| API Key | Secret token presented in the HTTP Authorization header to authorize every REST API request. |

# Appendix B: Analysis Models

## B.1 UML Use Case Diagram

Figure 1 shows the UML use case diagram. The General User uploads a dataset (which always includes validation), selects a model type, runs fraud detection (which always includes the model selection), and views predictions, performance metrics and downloadable results. The Developer/Tester invokes the Web Server's REST API directly and tests individual model microservices through their own HTTP contract.

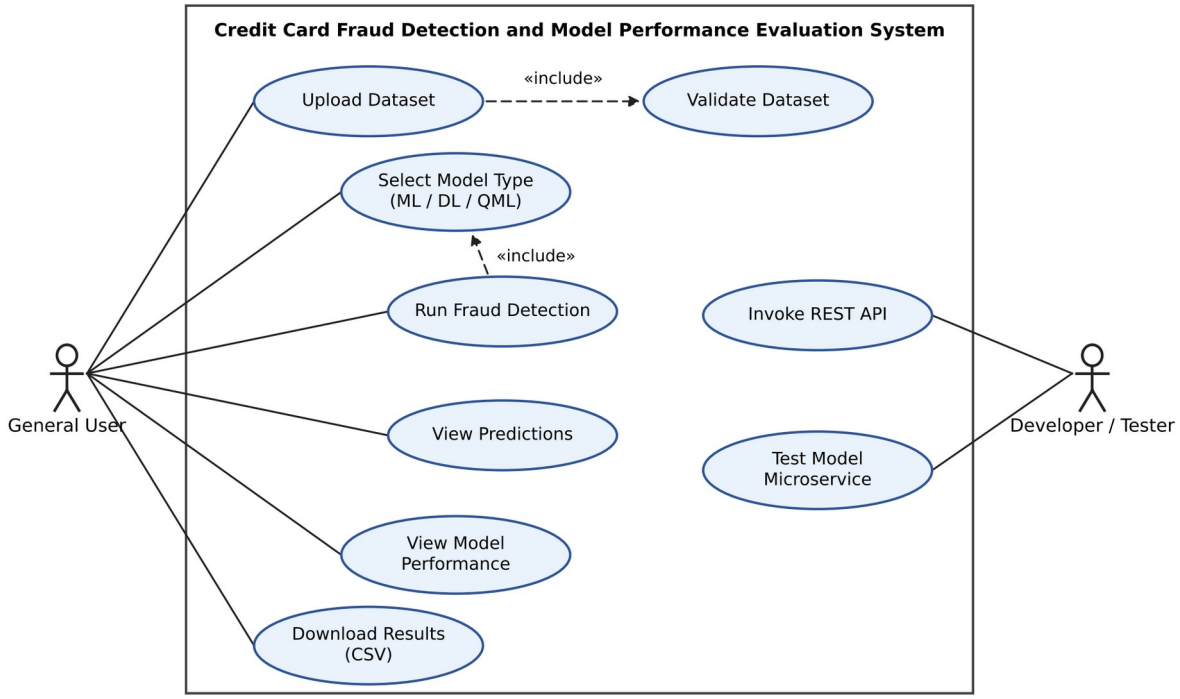



*Figure 1. UML use case diagram.*

| Use case | Actor | Requirements realized |
|---|---|---|
| Upload Dataset | General User | REQ-FD-01, REQ-FD-02, REQ-FD-05, REQ-API-01, SR-03 |
| Validate Dataset («include») | — | REQ-FD-03, REQ-FD-04, SR-01, BR-01, NFR-01, NFR-03, NFR-07 |
| Select Model Type | General User | REQ-FD-06 – REQ-FD-09, BR-02 |
| Run Fraud Detection | General User | REQ-FD-10 – REQ-FD-14, REQ-API-02 |
| View Predictions | General User | REQ-FD-13, REQ-FD-18, REQ-FD-20, REQ-UI-05 |
| View Model Performance | General User | REQ-FD-15 – REQ-FD-17, REQ-FD-19, BR-03 |
| Download Results (CSV) | General User | REQ-FD-21, SR-04 |
| Invoke REST API | Developer / Tester | REQ-API-01 – REQ-API-05 |
| Test Model Microservice | Developer / Tester | REQ-FD-10, NFR-Q05 |

## B.2 System Architecture Diagram

Figure 2 shows the three-tier architecture: the Web Client communicates with the Flask Web Server over HTTP/REST; the server authenticates and validates requests, stores datasets in the server-local dataset store, and sends the dataset identifier to the ML, DL or QML microservice; each microservice reads the dataset file from the store and exposes the uniform POST /predict / GET /health contract.

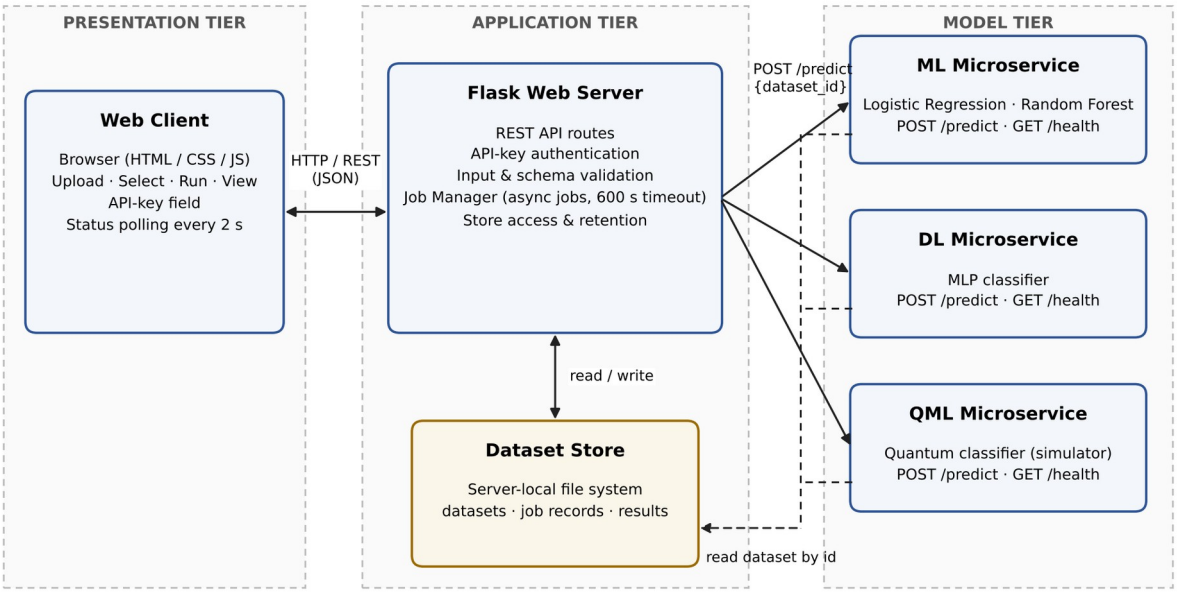



*Figure 2. High-level three-tier architecture.*

## Appendix C: To Be Determined List

| ID | Description | Status |
|---|---|---|
| TBD-01 | Deployment beyond a single host (microservices on other machines, a shared dataset store, containerization, process supervision). | Open |
| TBD-02 | Communication security for access from other machines (TLS termination, secret management for API keys). | Open |
| TBD-03 | Final selection of the quantum-circuit simulator backend for the QML microservice. | Open |

Resolved in v1.1: dataset file format and schema (REQ-FD-02, REQ-FD-03), baseline ML/DL/QML models (REQ-FD-07 – REQ-FD-09), model-performance metrics (REQ-FD-15, REQ-FD-16), quantitative response-time and capacity targets (NFR-01 – NFR-05) and the API authentication mechanism (SR-03). Clarified in v1.2: upload timing boundaries (NFR-01, NFR-03, NFR-07), dataset access by the microservices (Section 3.3), results-retrieval authorization (SR-03), data display policy (SR-04) and retention (SR-05).

## Project Team

| SRN | Name |
|---|---|
| PES1UG24AM024 | Akshat Agarwal |
| PES1UG24AM030 | Aman Priyansh |
| PES1UG24AM015 | Aditya Kalloli |
| PES1UG23AM224 | Raghav Balaji |